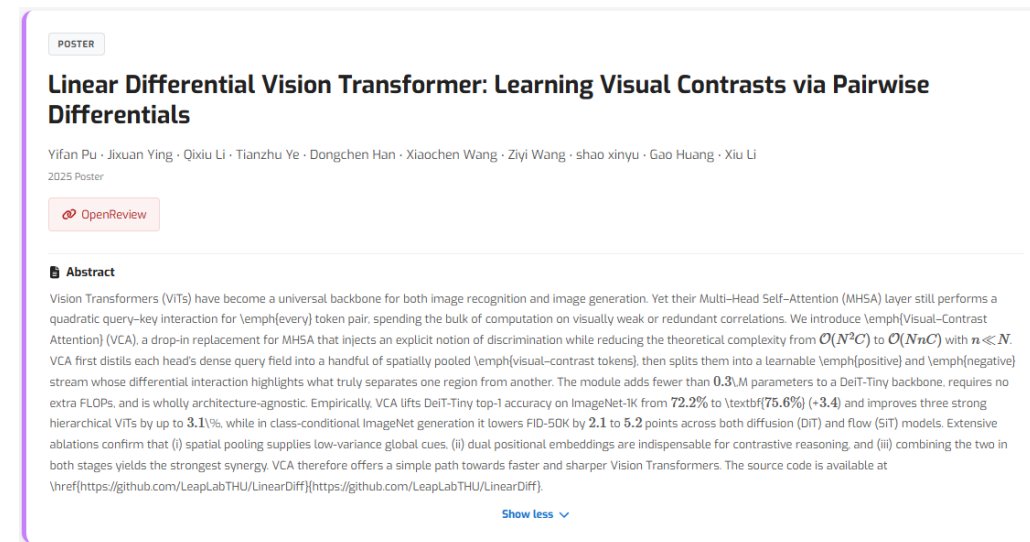


Linear Differential Vision Transformer: Learning Visual Contrasts via Pairwise Differentials

NeurIPS 2025
citations 7

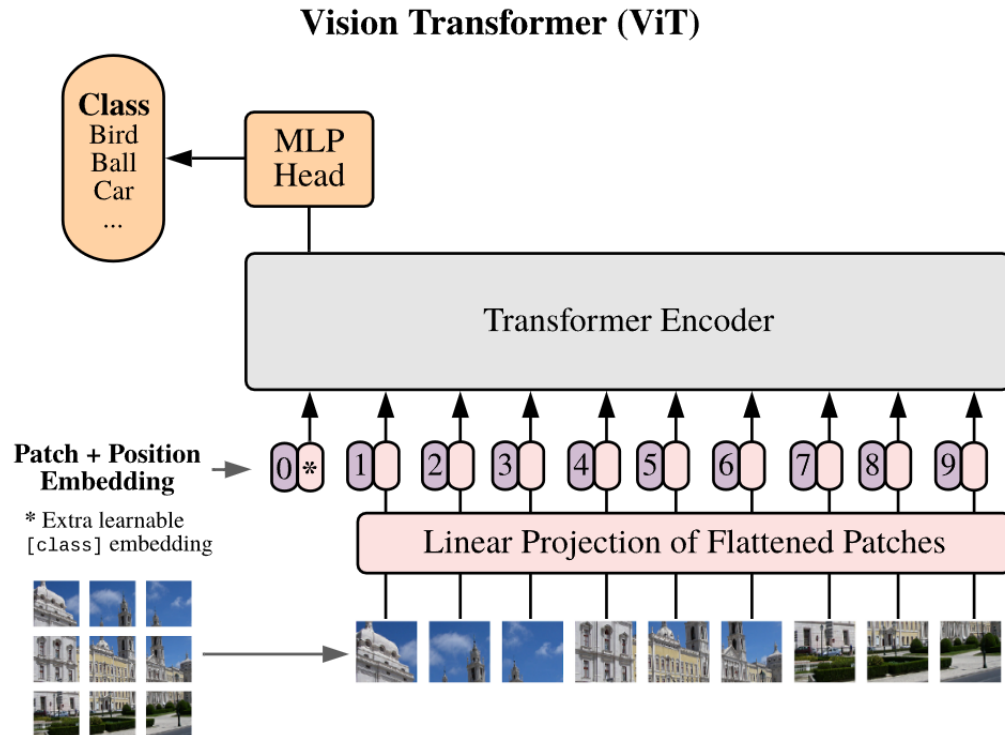
Department of Computer Engineering and Artificial Intelligence
MAI Lab.

YS Park

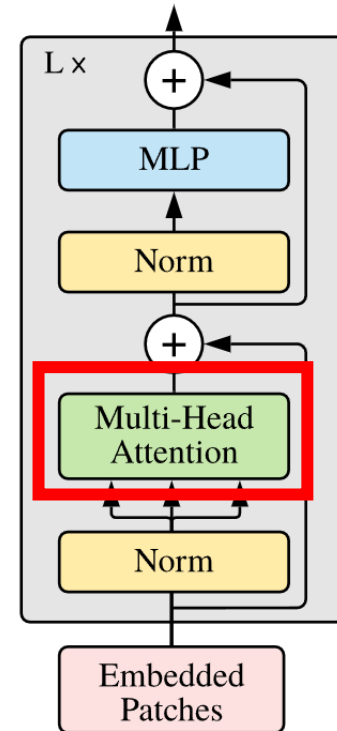


- What is Differential Attention?
- Introduction
- Approach
- Experiments
- Conclusion

What is Differential Attention?



Transformer Encoder



$$\text{self-attention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times D}$$

$$\mathbf{Q}\mathbf{K}^\top = \mathbf{A}_{N^2} \text{ and } \mathcal{O}(N^2)$$

Where N is the # of tokens

DIFFERENTIAL TRANSFORMER

Tianzhu Ye^{*†‡} Li Dong^{*†} Yuqing Xia^{*†} Yutao Sun^{*†‡}

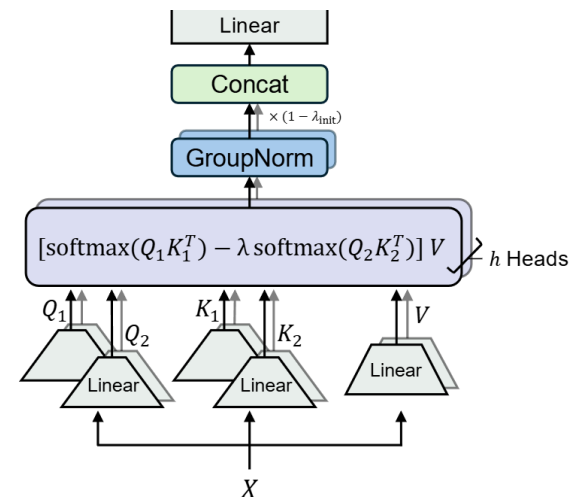
Yi Zhu[†] Gao Huang[‡] Furu Wei^{†◇}

[†] Microsoft Research [‡] Tsinghua University

<https://aka.ms/GeneralAI>

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

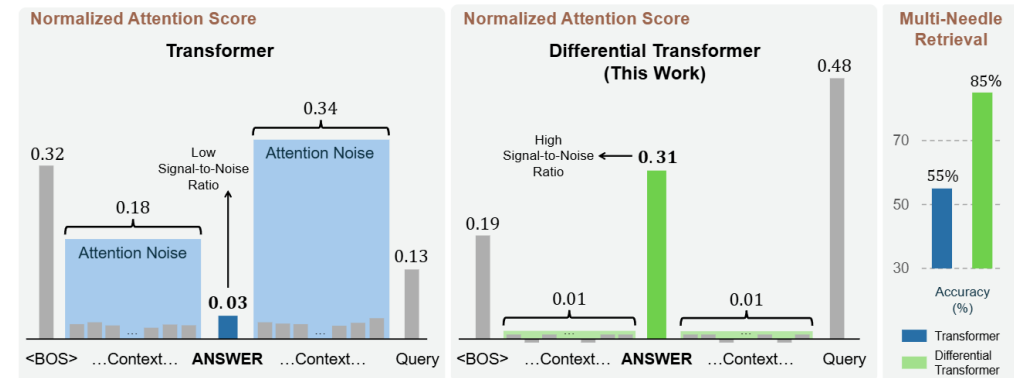
$$\text{DA} = (\mathbf{A}_1 - \lambda \mathbf{A}_2) \mathbf{V}$$



```

def DiffAttn(X, W_q, W_k, W_v, λ):
    Q1, Q2 = split(X @ W_q)
    K1, K2 = split(X @ W_k)
    V = X @ W_v
    # Qi, Ki: [b, n, d]; V: [b, n, 2d]
    s = 1 / sqrt(d)
    A1 = Q1 @ K1.transpose(-1, -2) * s
    A2 = Q2 @ K2.transpose(-1, -2) * s
    return
        (softmax(A1) - λ softmax(A2)) @ V

def MultiHead(X, W_q, W_k, W_v, W_o, λ):
    O = GroupNorm([DiffAttn(X, W_qi, W_ki,
        W_vi, λ) for i in range(h)])
    O = O * (1 - λ_init)
    return Concat(O) @ W_o
    
```



- 문제 1: ViT는 $N \times N$ 의 similarity matrix를 만들기 때문에 $\mathcal{O}(N^2C)$ 의 복잡도를 가짐.
 - 이미지 해상도가 높아지거나 patch 크기가 작아지면 N 이 빠르게 증가하므로, attention 비용이 전체 training과 inference 비용의 큰 부분을 차지함.
 - 기존 해결법
 - limiting the receptive field
 - low-rank projection / kernel approximation을 통해 attention map을 근사하는 방식
- 문제 2: 관계가 거의 의미가 없어도 계산한 값이 그저 작을 뿐임.
 - 이는 수렴을 느리게 하는 주요 원인임.
 - 기존 해결법
 - Differential Attention (DA)
- 문제 1과 문제 2를 동시에 해결할 수 있을까?
 - Image의 locality + DA의 메커니즘을 결합해보자!

$$\begin{aligned}
 q &= z^{l-1} W^q, k = z^{l-1} W^k, v = z^{l-1} W^v. \\
 z^{l-1} &\in \mathbb{R}^{N \times C}, W^q, W^k, W^v \in \mathbb{R}^{C \times C}, \\
 q, k, v &\in \mathbb{R}^{N \times C} \\
 q^{(m)}, k^{(m)}, v^{(m)} &\in \mathbb{R}^{N \times d}, d = \frac{C}{M},
 \end{aligned}$$

$$\begin{aligned}
 A^{(m)} &= \text{Softmax} \left(\frac{q^{(m)} k^{(m)\top}}{\sqrt{d}} \right) \\
 A^{(m)} &\in \mathbb{R}^{N \times N} \\
 h^{(m)} &= A^{(m)} v^{(m)} \in \mathbb{R}^{N \times d}
 \end{aligned}$$

$$\begin{aligned}
 h &= \text{Concat}(h^{(1)}, \dots, h^{(M)}), z^l = h W^o \\
 h &\in \mathbb{R}^{N \times C}, W^o \in \mathbb{R}^{C \times C}
 \end{aligned}$$

```

class Attention(nn.Module):
    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None, attn_drop=0., proj_drop=0.):
        super().__init__()
        self.num_heads = num_heads
        head_dim = dim // num_heads
        self.scale = qk_scale or head_dim ** -0.5

        self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop)

    def forward(self, x):
        B, N, C = x.shape
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, C // self.num_heads).permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]

        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        x = (attn @ v).transpose(1, 2).reshape(B, N, C)
        x = self.proj(x)
        x = self.proj_drop(x)
        return x, attn
    
```

$O(N^2C)$

- 이 과정에서 $A^{(m)}$ 가 $N \times N$ 행렬이므로, 계산 복잡도는 대략 $O(N^2C)$ 임.

https://github.com/facebookresearch/dino/blob/main/vision_transformer.py

Approach – Differential Attention

$$[q_1; q_2] = z^{l-1} [W_1^q; W_2^q], [k_1; k_2] = z^{l-1} [W_1^k; W_2^k],$$

$$v = z^{l-1} W^v$$

$$W_1^q, W_2^q, W_1^k, W_2^k \in \mathbb{R}^{(C/2) \times C}, q_1, q_2, k_1, k_2 \in \mathbb{R}^{N \times (\frac{C}{2})},$$

$$W^v \in \mathbb{R}^{C \times C}, v \in \mathbb{R}^{N \times C}$$

$$q_1^{(m)}, q_2^{(m)}, k_1^{(m)}, k_2^{(m)} \in \mathbb{R}^{N \times (d/2)}, v^{(m)} \in \mathbb{R}^{N \times d}$$

$$A_1^{(m)} = \text{Softmax} \left(\frac{q_1^{(m)} k_1^{(m)T}}{\sqrt{d/2}} \right),$$

$$A_2^{(m)} = \text{Softmax} \left(\frac{q_2^{(m)} k_2^{(m)T}}{\sqrt{d/2}} \right)$$

$$A^{(m)} = A_1^{(m)} - \lambda A_2^{(m)},$$

$$\lambda = \exp(\lambda_1^q \cdot \lambda_1^k) - \exp(\lambda_2^q \cdot \lambda_2^k) + \lambda_{\text{init}}$$

$$\hat{h}^{(m)} = A^{(m)} v^{(m)},$$

$$h^{(m)} = (1 - \lambda_{\text{init}}) \text{RMSNorm}(\hat{h}^{(m)})$$

$$h = \text{Concat}(h^{(1)}, \dots, h^{(M)}), z^l = h W^O$$

$$h \in \mathbb{R}^{N \times C}, W^O \in \mathbb{R}^{C \times C}$$

<https://github.com/paulilioaica/Differential-Transformer/blob/main/src/transformer.py>

```

class MultiHeadAttention(nn.Module):
    def __init__(self, num_hidden, num_heads, seq_len, d_k) -> None:
        super().__init__()
        self.num_hidden = num_hidden
        self.num_heads = num_heads
        self.seq_len = seq_len
        self.d_k = torch.tensor(d_k)

        self.W_q = nn.Linear(num_hidden, 2 * num_heads * num_hidden)
        self.W_k = nn.Linear(num_hidden, 2 * num_heads * num_hidden)
        self.W_v = nn.Linear(num_hidden, num_heads * num_hidden)
        self.W_o = nn.Linear(num_heads * num_hidden, num_hidden)

        self.softmax = nn.Softmax(dim=-1)
        self.dropout = nn.Dropout(0.1)
        self.group_norm = nn.GroupNorm(num_groups=num_heads, num_channels=num_heads)
        self._lambda_init = torch.rand(1)
        self._lambda = nn.Parameter(self._lambda_init.clone())

    def forward(self, query, key, values, dropout=0.1, mask=None):
        query = self.W_q(query).view(-1, self.num_heads, self.seq_len, 2 * self.num_hidden)
        key = self.W_k(key).view(-1, self.num_heads, self.seq_len, 2 * self.num_hidden)
        values = self.W_v(values).view(-1, self.num_heads, self.seq_len, self.num_hidden)

        #split query into [q1;q2] and same for keys [k1;k2]
        query_1 = query[:, :, :, :self.num_hidden]
        query_2 = query[:, :, :, self.num_hidden:]

        key_1 = key[:, :, :, :self.num_hidden]
        key_2 = key[:, :, :, self.num_hidden:]

        QK_T_1 = torch.matmul(query_1, key_1.mT) / torch.sqrt(self.d_k)
        QK_T_2 = torch.matmul(query_2, key_2.mT) / torch.sqrt(self.d_k)

        QK_T_1_norm = self.softmax(QK_T_1)
        QK_T_2_norm = self.softmax(QK_T_2)

        #eq 1
        attention_scores = (QK_T_1_norm - self._lambda * QK_T_2_norm)
        attention_scores = self.dropout(attention_scores)
        output = torch.matmul(attention_scores, values)
        output = output.transpose(1, 2).contiguous().view(-1, self.num_heads, self.seq_len, self.num_hidden)

        output = self.group_norm(output)

        output = output * (1 - self._lambda_init)
        output = torch.cat([output[:, i, :, :] for i in range(self.num_heads)], dim=-1)

        output = self.W_o(output)
        return output
    
```

- Vision 특화 + $O(N^2C)$ 완화를 위해, 각 head 별로 **visual contrast tokens**를 도입한 후 2가지 stage로 Attention을 분해/구성함.
- **Stage I: Global Contrast**

$$\begin{aligned}t_+^{(m)}, t_-^{(m)} &\in \mathbb{R}^{n \times d}, \\k^{(m)}, v^{(m)} &\in \mathbb{R}^{N \times d}, \\ \hat{v}_+^{(m)}, \hat{v}_-^{(m)} &\in \mathbb{R}^{n \times d}\end{aligned}$$

$$v_+^{(m)} = \text{Softmax}\left(\frac{t_+^{(m)} k^{(m)\top}}{\sqrt{d}}\right) v^{(m)}, v_-^{(m)} = \text{Softmax}\left(\frac{t_-^{(m)} k^{(m)\top}}{\sqrt{d}}\right) v^{(m)}$$

$$\begin{aligned}\hat{v}^{(m)} &= \left(1 - \lambda_{\text{init}}^{(1)}\right) \text{RMSNorm}\left(\hat{v}_+^{(m)} - \lambda^{(1)} \hat{v}_-^{(m)}\right), \\ \lambda^{(1)} &= \exp\left(\lambda_{q_1}^{(1)} \cdot \lambda_{k_1}^{(1)}\right) - \exp\left(\lambda_{q_2}^{(1)} \cdot \lambda_{k_2}^{(1)}\right) + \lambda_{\text{init}}^{(1)}\end{aligned}$$

- Vision 특화 + $O(N^2C)$ 완화를 위해, 각 head 별로 **visual contrast tokens**를 도입한 후 2가지 stage로 Attention을 분해/구성함.
- **Stage II: Patch-wise Differential Attention**

$$q^{(m)} \in \mathbb{R}^{N \times d},$$
$$A_1^{(m)}, A_2^{(m)} \in \mathbb{R}^{N \times n}$$

$$A_1^{(m)} = \text{Softmax} \left(\frac{q^{(m)} t_+^{(m)^\top}}{\sqrt{d}} \right), A_2^{(m)} = \text{Softmax} \left(\frac{q^{(m)} t_-^{(m)^\top}}{\sqrt{d}} \right)$$

$$A^{(m)} = A_1^{(m)} - \lambda^{(2)} A_2^{(m)},$$
$$\lambda^{(2)} = \exp \left(\lambda_{q_1}^{(2)} \cdot \lambda_{k_1}^{(2)} \right) - \exp \left(\lambda_{q_2}^{(2)} \cdot \lambda_{k_2}^{(2)} \right) + \lambda_{\text{init}}^{(2)}$$

$$h^{(m)} = A^{(m)} v^{(m)}$$
$$h^{(m)} = \left(1 - \lambda_{\text{init}}^{(2)} \right) \text{RMSNorm}(h^{(m)})$$

- Vision 특화 + $O(N^2C)$ 완화를 위해, 각 head 별로 **visual contrast tokens**를 도입한 후 2가지 stage로 Attention을 분해/구성함.
- **Visual Contrast Token Generation**
 - 각 head의 query token $q^{(m)}$ 을 2차원 이미지 feature map으로 되돌린 뒤, spatial average pooling을 적용해 compact visual-contrast token을 생성함.

$$q^{(m)} \in \mathbb{R}^{N \times d},$$

$$q^{(m)} \rightarrow \tilde{q}^{(m)} \in \mathbb{R}^{H \times W \times d}, \quad HW = N,$$

$$\tilde{t}^{(m)} = \text{AvgPool}(\tilde{q}^{(m)}) \in \mathbb{R}^{h \times w \times d},$$

$$\tilde{t}_+^{(m)} = \tilde{t}^{(m)} + e_+,$$

$$\tilde{t}_-^{(m)} = \tilde{t}^{(m)} + e_-,$$

$$t_+^{(m)} = \text{Flatten}(\tilde{t}_+^{(m)}),$$

$$t_-^{(m)} = \text{Flatten}(\tilde{t}_-^{(m)}),$$

$$t_+^{(m)}, t_-^{(m)} \in \mathbb{R}^{n \times d}, \quad n = hw \ll N$$

- Datasets: ImageNet-1K
 - image recognition: 224×224 , The top-1 accuracy on the validation set
 - image generation: 256×256 , FID-50K
- Training Configuration:
 - Image Recognition:
 - Optimizer: AdamW (Weight decay: 0.05)
 - 학습: 300 epochs
 - Learning rate: 1×10^{-3} (LR schedule: 20epochs linear warm-up 후 cosine decay)
 - Augmentation/regularization: RandAugment, Mixup, CutMix, Random Erasing
 - EMA: CSwin 계열 설정을 따름
- Image Generation (DiT/SiT)
 - Optimizer: AdamW
 - 학습: 400,000iterations (Global batch size: 256)
 - Learning rate: 1×10^{-4} , constant
 - EMA Decay rate: 0.999

Table 1: Image classification results on ImageNet-1K

Method	#Params	FLOPs	Top-1 Acc.	Method	#Params	FLOPs	Top-1 Acc.
DeiT-T	5.7M	1.2G	72.2	Swin-T	28.9M	4.5G	81.3
+Ours	6.0M	1.2G	75.6 ($\uparrow 3.4$)	+Ours	28.5M	4.6G	82.3 ($\uparrow 1.0$)
DeiT-S	22.1M	4.6G	79.8	Swin-S	49.7M	8.7G	83.0
+Ours	22.6M	4.6G	80.7 ($\uparrow 0.9$)	+Ours	49.6M	8.7G	83.7 ($\uparrow 0.7$)
PVT-T	13.2M	1.9G	75.1	Swin-B	88.1M	15.4G	83.5
+Ours	11.6M	2.0G	78.2 ($\uparrow 3.1$)	+Ours	87.9M	15.5G	83.9 ($\uparrow 0.4$)
PVT-S	24.5M	3.8G	79.8	CSwin-T	20.5M	4.3G	82.7
+Ours	20.6M	4.1G	82.3 ($\uparrow 2.5$)	+Ours	20.4M	4.3G	83.3 ($\uparrow 0.6$)
PVT-M	35.9M	7.0G	81.2	CSwin-S	32.8M	6.8G	83.6
+Ours	35.8M	7.2G	83.2 ($\uparrow 2.0$)	+Ours	32.7M	6.8G	84.0 ($\uparrow 0.4$)

Table 2: Class-conditional image generation results on ImageNet-1K with 256×256 resolution.

Method	#Params	FLOPs	FID-50K(↓)	Method	#Params	FLOPs	FID-50K(↓)
DiT-S/8	33.0M	0.4G	151.9	SiT-S/8	33.0M	0.4G	149.5
+Ours	33.8M	0.4G	148.3 (↓3.6)	+Ours	33.0M	0.4G	147.4 (↓2.1)
DiT-S/4	32.9M	1.4G	97.9	SiT-S/4	32.9M	1.4G	84.0
+Ours	33.6M	1.5G	92.7 (↓5.2)	+Ours	33.6M	1.5G	80.9 (↓3.1)
DiT-S/2	33.0M	6.1G	67.2	SiT-S/2	33.0M	6.1G	57.3
+Ours	33.6M	6.0G	62.3 (↓4.9)	+Ours	33.6M	6.0G	53.0 (↓4.3)
DiT-B/8	130.7M	1.4G	118.4	SiT-B/8	130.7M	1.4G	106.0
+Ours	132.1M	1.5G	114.4 (↓4.0)	+Ours	132.1M	1.5G	102.1 (↓3.9)
DiT-B/4	130.4M	5.6G	68.3	SiT-B/4	130.4M	5.6G	55.9
+Ours	131.8M	5.8G	66.0 (↓2.3)	+Ours	131.8M	5.8G	53.6 (↓2.3)
DiT-B/2	130.5M	23.0G	42.9	SiT-B/2	130.5M	23.0G	35.3
+Ours	131.8M	22.9G	38.9 (↓4.0)	+Ours	131.8M	22.9G	32.7 (↓2.6)

Table 3: Ablation on detailed model architectures across image classification and generation tasks.

Attention Type		Image Classification on DeiT-Tiny			Image Generation on DiT-S/2		
Stage I	Stage II	Params	FLOPs	Top-1 Acc.(↑)	Params	FLOPs	FID-50K(↓)
Ours	Vani.	6.0M	1.2G	75.5	33.6M	5.9G	64.3
Vani.	Ours	6.0M	1.2G	75.4	33.6M	5.9G	64.6
Diff.	Diff.	5.7M	1.2G	75.1	33.0M	5.8G	63.9
Ours	Ours	6.0M	1.2G	75.6	33.6M	6.0G	62.3

Table 4: Ablation on visual contrast token generation across image classification and generation tasks.

Token Type		Image Classification on DeiT-Tiny			Image Generation on DiT-S/2		
Pos. Str.	Neg. Str.	Params	FLOPs	Top-1 Acc.	Params	FLOPs	FID-50K
Emb.	Emb.	6.0M	1.2G	75.1	33.6M	6.0G	63.7
Pool	Pool+Emb.	5.9M	1.2G	75.5	33.3M	6.0G	64.1
Pool+Emb.	Pool	5.9M	1.2G	75.3	33.3M	6.0G	63.5
Pool+Emb.	Pool+Emb.	6.0M	1.2G	75.6	33.6M	6.0G	62.3

- Visual-Contrast Attention(VCA) 는 기존 MHSA를 대체할 수 있는 plug-and-play 모듈임.
- 이미지를 소수의 pooled token으로 요약한 뒤, 이를 pos/neg stream으로 나누고 차이를 계산함.
 - 이를 통해 중요한 시각적 관계는 **강조**하고, 중복되거나 덜 중요한 관계는 **억제**함.
- Self-attention의 복잡도를 **linear complexity**로 줄이면서도 추가 파라미터와 FLOPs가 적음.
 - 일반 ViT와 hierarchical ViT 모두에서 image classification 성능을 높였으며, diffusion 및 flow 기반 image generation의 품질도 개선함 .
- Attention을 **단순한 유사도 계산**이 아니라 명시적인 **contrast**를 학습하는 과정으로 볼 수 있다는 관점을 제시함.
- 한계:
 - Average pooling의 정보 손실
 - 작은 이미지에서 효율성 감소
 - 다른 분야로의 확장 미검증